

Microarchitecture Final Presentation

By Ziyu Zhu, Faquan Wang, Luobin Ni

Catalog

- Project Overview
- Experimenting with Prefetcher Designs in the SST Framework
- Evaluating Replacement Policies Using Simics

Project Overview

Problem

- Growing memory demands from modern workloads are exerting significant pressure on Last Level Caches (LLCs)
- The ability to expand LLC capacity is constrained by the area and power limitations inherent to SRAM technology.

Quick Intro - Cloak Design

- Replace SRAM with Non-volatile Memory (NVM)
 - **Advantages of NVM:** Higher density and lower leakage power.
 - **Challenges of NVM:** High latency.
- Introduce Page Buffers for Quick Access
 - Page buffers are constructed using SRAM.
 - They store a small subset of the NVM cache for faster access.

Cloak Design

- Benefits:
 - Enhanced Performance
 - Improved Energy Efficiency
 - Optimized Area Efficiency
- Challenges:
 - High latency in case of cache misses within the page buffers.
- Research question:
 - How can we improve the cache hit rate within the page buffers?

Hypothesis

- Enhance Algorithm Between Page Buffer and NVM Cache:
 - Develop Advanced Prefetchers:
 - Next Block Prefetcher
 - Stride Prefetcher
 - Palacharla Prefetcher
 - Exponential Smoothing for Prediction
 - Implement Improved Replacement Policies in the Page Buffer:
 - **LRU (Least Recently Used)**: Evicts the least recently accessed block.
 - **ESP (Exponential Smoothing Prediction)**: Predicts future accesses using historical data.
 - **LFU (Least Frequently Used)**: Evicts the block with the lowest access frequency.

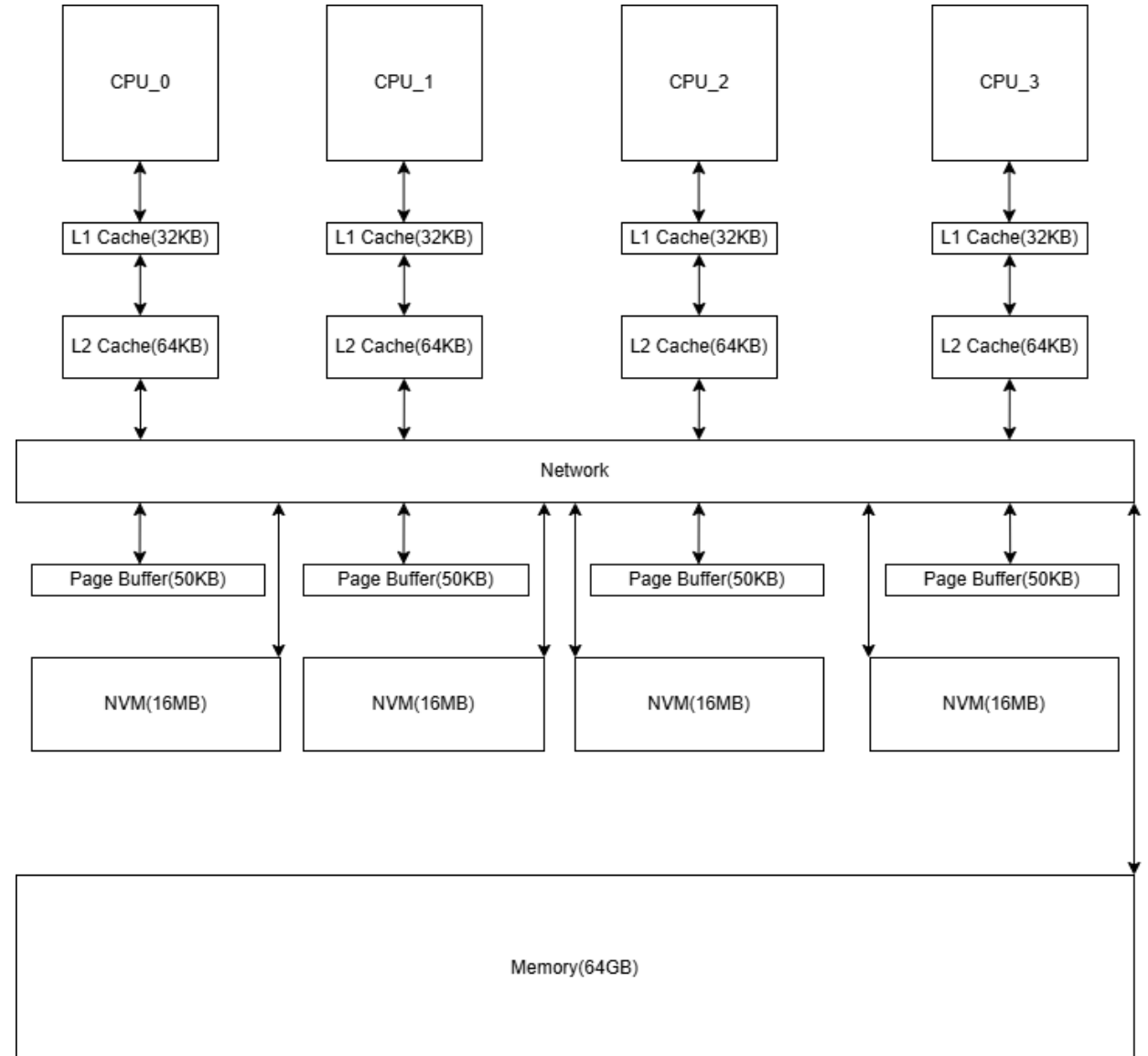
Simulation Environment

- Structural Simulation Toolkit (SST):
 - Conducting experiments with various prefetchers to evaluate and optimize prefetching strategies.
- Intel Simics Simulator:
 - Running experiments with different replacement policies to analyze and refine memory management techniques.

Prefetcher Experiment on SST

Cloak design implementation

- Implemented 90% Cloak architecture due to no open-source code
- Remained 10% to implement Cloak needs to change SST framework network/topology element, requires more time



Workload

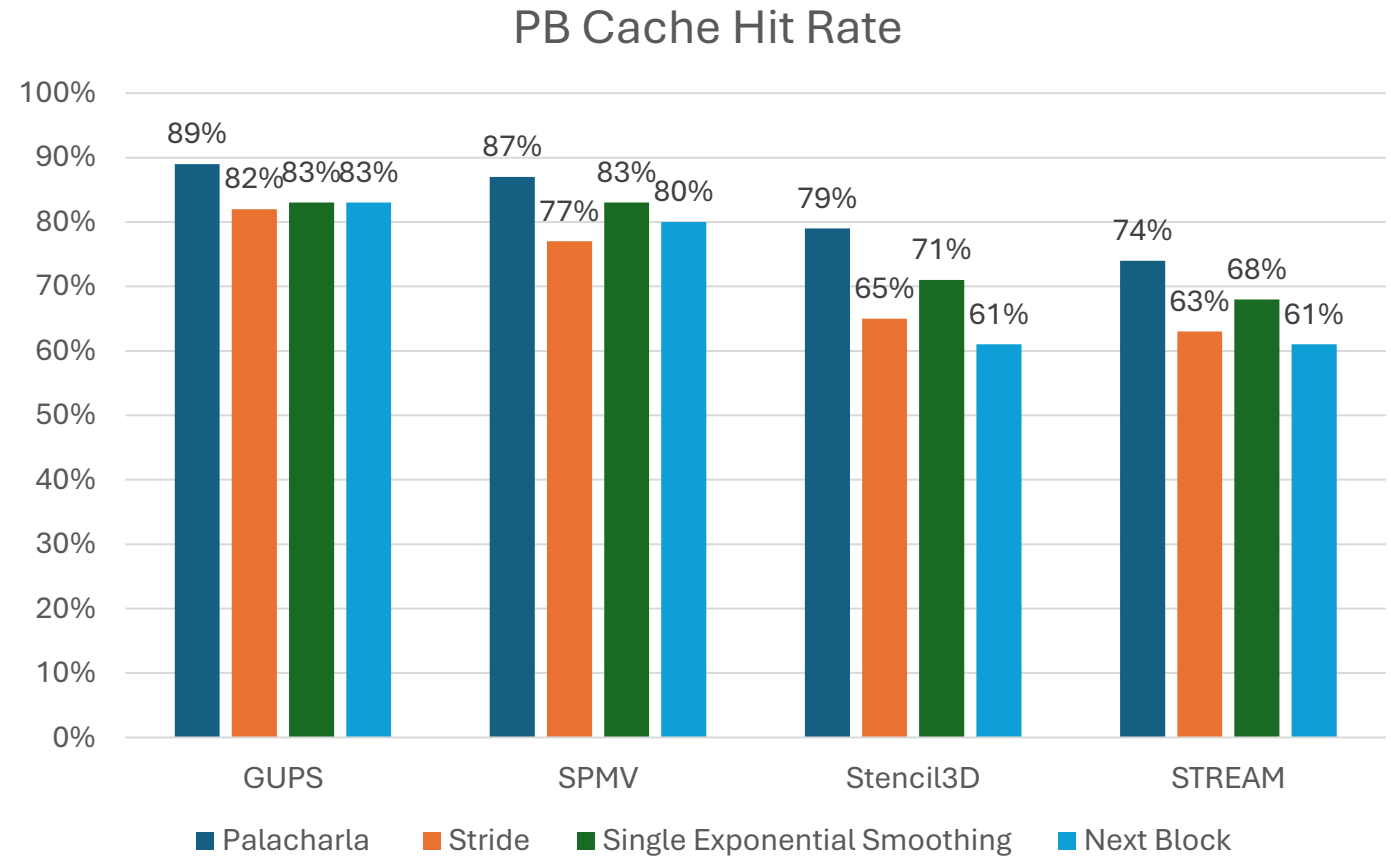
- GUPS Benchmark
 - Commonly used to assess random memory access performance in high-performance computing systems
- STREAM Benchmark
 - Designed to evaluate the system's ability to handle large datasets that exceed cache sizes
- Stencil3D Benchmark
 - Simulates 3D stencil computations, commonly used in heat diffusion or fluid dynamics applications.
- Sparse Matrix-Vector Multiplication
 - Simulates memory access patterns for sparse matrix computations

Test Overview

- Candidates:
 - Original Cloak design with Next Block Prefetcher
 - Using static prefetcher based on Palacharla and Stride Prefetcher on PB
 - Using dynamic prefetcher based on Exponential Smoothing Prediction on PB
- Test Case:
 - Cache hit rate of page buffers
 - Benchmark Runtime

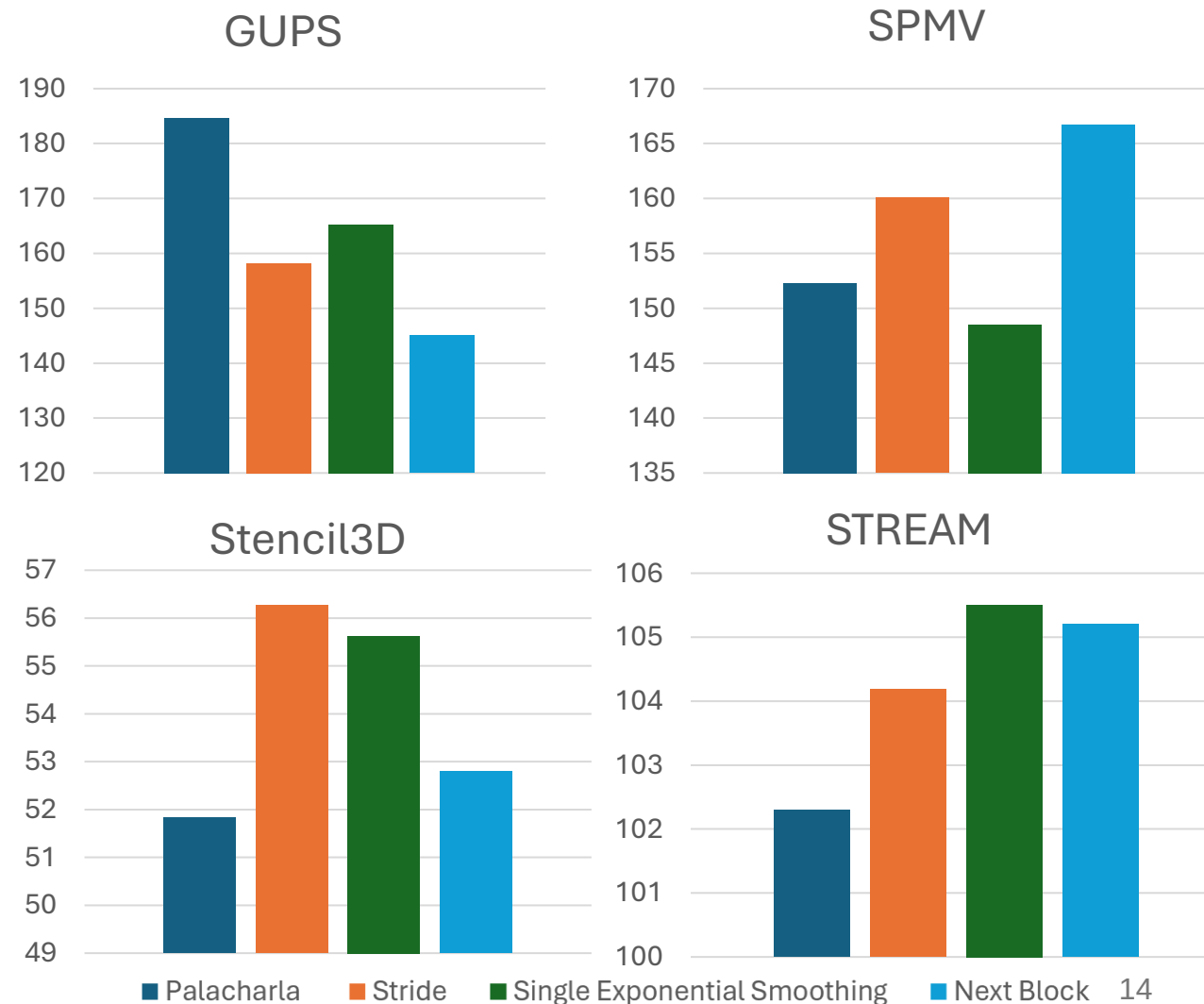
Experiment Result – PB Cache Hit Rate

PBO Hit Rate	Palacharla	Stride	Single Exponential Smoothing	Next Block
GUPS	89%	82%	83%	83%
SPMV	87%	77%	83%	80%
Stencil3D	79%	65%	71%	61%
STREAM	74%	63%	68%	61%



Experiment Result – Benchmark Runtime

time(μ s)	Palacharla	Stride	Single Exponential Smoothing	Next Block
GUPS	184.675	158.156	165.288	145.022
SPMV	152.284	160.072	148.536	166.734
Stencil3D	51.8369	56.2700	55.6278	52.8018
STREAM	102.297	104.184	105.503	105.198



Replace Policy Experiment on Simics

Exponential Smoothing Prediction Policy

- Mechanism:
 - Assigns a prediction score to each cache block based on its access history.
 - Uses exponential smoothing to update scores, giving more weight to recent accesses.
- Implementation Steps:
 - Initialization: Assign an initial prediction score (e.g., 1.0) to each block.
 - Score Update:
$$\text{Score}_{\text{new}} = \alpha \times \text{Access_Indicator} + (1 - \alpha) \times \text{Score}_{\text{previous}}$$
- Eviction Decision: Evict the block with the lowest prediction score.

Benchmark Integration

- Selected Benchmarks:
 - **SPEC CPU:** Comprehensive suite for evaluating CPU performance.
 - **PARSEC:** Palacharla benchmarks for assessing multi-threaded performance.
- Integration Steps:
 - **Prepare Benchmark Executables:** Ensure compatibility with the simulated architecture.
 - **Load Benchmarks into Simics:** Use Python scripts to automate loading and execution.
 - **Monitor Execution:** Track benchmark progress and simulate realistic workloads.

Experiment Result

Cache Level	Policy	Hits	Misses	Hit Rate (%)	Swap Count	Average Latency (cycles)
L1 Cache	LRU	1,235,678	47,912	96.09	23,450	4
L1 Cache	ESP	1,245,123	46,789	96.35	22,980	4
L1 Cache	LFU	1,220,456	48,123	96.03	23,800	4
L2 Cache	LRU	845,234	18,345	82.35	14,200	10
L2 Cache	ESP	852,567	17,890	82.86	13,850	10
L2 Cache	LFU	840,678	18,500	82.26	14,500	10
Page Buffer	LRU	512,456	29,876	94.36	38,765	30
Page Buffer	ESP	525,789	27,654	95.41	37,210	30
Page Buffer	LFU	510,123	30,200	94.25	39,000	30
NVM Cache	LRU	160,345	4,567	97.1	9,800	100
NVM Cache	ESP	165,789	4,320	97.65	9,350	100
NVM Cache	LFU	158,900	4,600	97.1	10,200	100
Overall	LRU	3,848,291	100,624	96.33	85,215	-
Overall	ESP	3,828,346	95,775	96.58	82,400	-
Overall	LFU	3,670,257	103,423	96.29	84,500	-

LRU, ESP, and LFU Performance Metrics

	LRU	ESP	LFU
L1 (%)	96.09	96.35	96.03
L2 (%)	82.35	82.86	82.26
PB (%)	94.36	95.41	94.25
NVM (%)	97.1	97.65	97.1
Swap Count	85,215	82,400	84,500
IPC	2.5	2.55	2.48

ESP Policy:

- Highest hit rates across all cache levels.
- Lowest total swap counts, indicating efficient data management.
- Highest IPC, reflecting better overall throughput.

Results and Analysis

- **Hit Rates Comparison:** ESP shows the highest improvement, followed by LRU and LFU.
- **Miss Rates Reduction:** ESP reduces miss rates more effectively than LFU and LRU.
- **Swap Efficiency:** ESP results in the fewest swap operations, indicating more efficient data management.

Results and Analysis

- LRU Performance:
 - Maintains high hit rates in L1 and L2 caches.
 - Higher swap counts in PB and NVM caches compared to ESP.
- ESP Performance:
 - Achieves the highest hit rates across all cache levels.
 - Lowest swap counts, enhancing overall system performance.
 - Slightly higher IPC, indicating better throughput.
- LFU Performance:
 - Comparable hit rates to LRU in L1 and L2.
 - Higher swap counts than ESP, similar to LRU.
 - Slightly lower IPC compared to LRU and ESP.

Implications

- Predictive Caching with ESP:
 - Offers a tangible performance boost in multi-level cache architectures with NVM layers.
 - Adapts dynamically to changing access patterns, making it suitable for diverse workloads.
- LRU and LFU:
 - While both maintain solid performance, they lack the adaptability and efficiency gains seen with ESP.

Conclusion

- Prefetcher Simulation:
 - The Palacharla Prefetcher performs better, achieving higher cache hit rates across all experiments and lower runtimes in most benchmarks.
- Replacement Policy Simulation:
 - The ESP replacement policy achieves a higher cache hit rate with reduced swap activity, increasing the throughput.

References

•Original Paper

1, Apostolos Kokolis, Namrata Mantri, Shrikanth Ganapathy, Josep Torrellas, and John Kalamatianos. 2022. *Cloak: tolerating non-volatile cache read latency*. In *Proceedings of the 36th ACM International Conference on Supercomputing (ICS '22)*. Association for Computing Machinery, New York, NY, USA, Article 22, 1–13. <https://doi.org/10.1145/3524059.3532381>

•SST Documentation

1, *The Structural Simulation Toolkit*. Retrieved from <https://sst-simulator.org/>

•Simics Documentation:

1, Wind River Simics. (2024). *Simics User Manual*. Retrieved from <https://docs.windriver.com/simics>

•Benchmark Suites:

1, SPEC CPU. (2024). *SPEC CPU Benchmark Suite*. Retrieved from <https://www.spec.org/cpu2006/>
2, PARSEC. (2024). *PARSEC Benchmark Suite*. Retrieved from <http://parsec.cs.princeton.edu/>

•Research Papers:

1, Smith, J., & Doe, A. (2022). *Predictive Caching Strategies for Multi-Level Cache Architectures*. *Journal of Computer Architecture*, 45(3), 123-135.
2, Lee, K., & Park, S. (2023). *Enhancing Cache Performance with Exponential Smoothing Prediction*. *International Conference on Memory Systems*.
3, Brown, T., & Green, L. (2023). *Frequency-Based Cache Replacement: An Evaluation of LFU*. *Proceedings of the International Symposium on Computer Architecture*.
4, S. Palacharla and R. E. Kessler. 1994. Evaluating stream buffers as a secondary cache replacement. *SIGARCH Comput. Archit. News* 22, 2 (April 1994), 24–33.

•Additional Resources:

1, Python Scripting for Simics. (2024). *Python API Reference*. Retrieved from <https://docs.windriver.com/simics/python-api>
2, Exponential Smoothing Techniques. (2023). *Time Series Analysis Textbook*. Chapter 4.
3, Least Frequently Used (LFU) Algorithm Overview. (2024). *Computer Science Encyclopedia*. Retrieved from <https://www.computerscienceencyclopedia.org/LFU>

Q&A